

Aula 03

Componente Curricular: Sistemas Web II

1. Objetivos de aprendizagem

- Compreender por que um teste unitário não deve depender de uma conexão real com o banco de dados.
- Utilizar o **MagicMock**, do módulo `unittest.mock`, para simular uma sessão (Session) do SQLAlchemy.
- Utilizar o mecanismo de sobrescrita de dependências do FastAPI (`dependency_overrides`) para injetar a sessão simulada nos testes.
- Escrever testes unitários para os endpoints GET e POST sem exigir um MySQL em execução.

2. Conteúdo programático

- Riscos de testar unitariamente contra um banco de dados real: lentidão, dependência de infraestrutura, dados que podem mudar entre execuções, risco de alterar dados reais.
- MagicMock: simulação do comportamento de um objeto (neste caso, a sessão do banco) sem executar nenhuma operação real.
- `app.dependency_overrides` do FastAPI: substituição de uma dependência (`get_db`) por uma versão simulada, apenas durante os testes.
- Verificação de chamadas em um mock (`assert_called_once`, entre outros).

3. Desenvolvimento da aula

3.1 Abertura

Retomar o CRUD completo com persistência real construído nas aulas anteriores e analisar: *"Agora que os endpoints dependem de uma sessão real do MySQL, como podemos testá-los automaticamente sem precisar de um banco de dados rodando a cada execução dos testes?"*

3.2 Por que não testar contra o banco real

Riscos de um teste unitário depender de um banco de dados real:

- O banco pode estar fora do ar, ou não configurado na máquina que executa os testes (por exemplo, em um servidor de integração contínua).
- Consultas reais ao banco tornam os testes mais lentos, especialmente quando há muitos testes.
- Dados cadastrados por um teste podem interferir no resultado de outro teste, ou alterar dados que não deveriam ser alterados.

Assim como na aula sobre mocks, a solução é isolar a unidade testada de sua dependência real — neste caso, substituindo a sessão do banco de dados por uma simulada.

3.3 MagicMock e dependency override

O MagicMock permite criar um objeto capaz de simular qualquer método ou atributo chamado nele, e o `dependency_overrides` do FastAPI, que permite substituir, apenas durante os testes, a função `get_db` por uma versão que devolve esse mock:

```
# test_produtos.py
from unittest.mock import MagicMock
from fastapi.testclient import TestClient
from main import app, get_db
from models import ProdutoDB

client = TestClient(app)
```

Com o MagicMock, é possível definir o que cada método deve retornar quando chamado — por exemplo, `db.query(ProdutoDB).all()` pode ser configurado para retornar uma lista de produtos de teste, sem consultar o banco de verdade.

3.4 Implementando os testes com mock da sessão

Teste do endpoint de listagem, simulando o retorno da consulta ao banco:

```
def test_listar_produtos_com_mock():
    db_mock = MagicMock()
    db_mock.query.return_value.all.return_value = [
        ProdutoDB(id=1, nome='Teclado', preco=89.90, quantidade=15)
    ]
    app.dependency_overrides[get_db] = lambda: db_mock
```

```
resposta = client.get('/produtos')

assert resposta.status_code == 200
assert resposta.json()[0]['nome'] == 'Teclado'

app.dependency_overrides.clear()
```

Em seguida, implementar o teste do endpoint de cadastro, verificando que os métodos add e commit da sessão foram realmente chamados pelo endpoint, sem que nenhum dado tenha sido gravado em um banco real:

```
def test_criar_produto_com_mock():
    db_mock = MagicMock()

    def simular_refresh(produto):
        produto.id = 1 # simula o banco atribuindo um id ao registro

    db_mock.refresh.side_effect = simular_refresh
    app.dependency_overrides[get_db] = lambda: db_mock

    novo_produto = {'nome': 'Monitor', 'preco': 799.90, 'quantidade': 5}
    resposta = client.post('/produtos', json=novo_produto)

    assert resposta.status_code == 201
    db_mock.add.assert_called_once()
    db_mock.commit.assert_called_once()

    app.dependency_overrides.clear()
```

Executar "**pytest -v**" e destacar dois pontos importantes:

- os testes rodam instantaneamente, sem qualquer conexão com o MySQL;
- e **app.dependency_overrides.clear()** é essencial ao final de cada teste, para que o mock de um teste não interfira no próximo.

3.5 Atividade prática

Cada aluno deve escrever, para a API do recurso próprio (criada na aula 2), testes unitários com mock da sessão do banco para os endpoints de listagem e cadastro, seguindo o mesmo padrão construído em aula.

4. Referências

- Documentação oficial do FastAPI — Testing Dependencies with Overrides:
<https://fastapi.tiangolo.com/advanced/testing-dependencies/>
- Documentação oficial do Python — unittest.mock:
<https://docs.python.org/3/library/unittest.mock.html>
- Material e exemplos de código disponibilizados em aula.